

ShopSpark

1. Industry Relevance

Online retail in the United States runs on attention. A typical shopper sees dozens of product slots on a homepage, category page, or cart page in a single session. Most of those slots get ignored. The few that are clicked are the ones that decide whether the shopper stays, buys, or leaves. Click-through rate, or CTR, is the metric that captures this directly: of all the products we showed, how many did the shopper click on.

For mid-size US e-commerce companies, lifting the CTR of recommendation slots by even one percentage point shows up clearly in revenue. Industry benchmarks from Triple Whale (which tracks over 30,000 US e-commerce brands) place the median paid CTR at around 1.77 percent. WebFX places the average Google Search CTR for e-commerce around 2.69 percent. On-site recommendation widgets, which serve customers who are already in a buying mindset, can reach 5 to 10 percent CTR when targeted well. Companies like Amazon, Wayfair, Target, Chewy, and Etsy all run dedicated teams to predict which product to show to which user, in which slot, at which moment.

This case is set in a mid-size US e-commerce company that sells across categories (electronics, fashion, home, beauty, grocery, sports, books, and toys) on its website and mobile app. The set-up is similar to companies like Wayfair, Chewy, or Bed Bath and Beyond at an earlier stage of their growth.

2. Business Context

ShopSpark Retail is a mid-size US e-commerce company that sells across eight product categories. It serves customers through a website and a mobile app, with mobile making up around 77 percent of traffic. The company has 8,000 active customers and a catalog of 704 products. Customers range from new sign-ups to loyal repeat buyers, spread across major US metros (New York, Los Angeles, Chicago, Houston, Dallas, Phoenix, Philadelphia, San Antonio, San Diego) and a long tail of smaller cities.

On every page a shopper visits, ShopSpark fills several recommendation slots with products. Today, these slots are filled using simple rules: best-sellers in the category, most-viewed in the last week, or items that are on discount. The product team believes the company is leaving money on the table. The same homepage carousel is shown to a 22-year-old fashion shopper in New York and to a 50-year-old home goods buyer in Phoenix. The team wants a data-driven way to predict which product is most likely to be clicked by which user in which context, so that, in a later phase, the ranking on each slot can be driven by that prediction.

Objective

To build a model that predicts the probability of a click for any product shown to any shopper in any context, so that recommendation slots across the website and app can be ranked by predicted click probability instead of today's static rules.

3. Problem Statement

Current State

Recommendation slots are filled by static rules. There is no view of which user is likely to click which product. The product team cannot personalize. Marketing and merchandising teams see flat or declining CTRs across most slots, but cannot explain why. There is no model in place and no scoring pipeline.

Future State

A trained CTR prediction model exists, with a documented feature pipeline and a clear view of model quality on a held-out test set. The product team can explain, in plain English, which features matter most and why a particular product gets a high or low predicted click probability. The model and its feature pipeline are ready to be handed over to engineering for the next phase, where the predictions can be used to rank products in recommendation slots.

The gap is the move from zero predictive scoring today to a working CTR prediction model, trained on real impression data, with documented model quality.

4. Context and Constraints

People

The product and marketing teams are used to working with dashboards and A/B test results, not predictive models. There is one data analyst and one junior data scientist on the team. Senior leadership is supportive but wants the first model to be simple to explain and easy to debug, not a black box.

Process

Impression logs are captured today but not analyzed in any structured way. There is no feature store, no model serving infrastructure, and no MLOps practice. The first version of the model will be trained offline, on a snapshot of the data. Anything beyond that (serving, retraining schedule, monitoring) is out of scope.

Platform

Data sits across five separate tables exported from the website event logger, the product catalog system, the user CRM, and the campaign management tool. There is no unified data warehouse yet. For now, the team will work in Python on a laptop with standard libraries (pandas, scikit-learn, XGBoost). The dataset (around 80,000 impressions) is small enough to fit in memory.

5. Past Attempts

About six months ago, the product team tried two things to lift CTR. Both helped a little, but not enough.

- Rule 1: Show top-5 best-sellers in the user's last-viewed category. This lifted CTR on the homepage by around 8 percent, but it failed for new users with no view history, and it ignored time-of-day and device.
- Rule 2: Promote discounted items in the top slot. This worked for price-sensitive shoppers but pushed away premium buyers, who started ignoring the carousel. Average order value from the carousel actually dropped.

The team concluded that single-rule heuristics are too crude. The signals that drive a click are many and they interact. A shopper's past CTR matters, but so does the slot position, the time of day, whether the product is in stock, whether it matches the shopper's favorite category, and so on. A model that can combine all these signals at once is the right next step.

6. Proposed Solution

The proposed solution is to bring the five data sources together, build a single training table at the impression level, engineer features that capture user behavior and context, and train three tree-based ensemble models (Random Forest, Gradient Boosting, XGBoost) on the binary click label. The three models will be compared on a held-out test set and the best one selected for handover.

Scope of Work

- Join the five source tables to build one impression-level training table.
- Study the data through EDA to understand distributions, missing values, class imbalance, and feature interactions.
- Clean and preprocess the data: handle missing values, encode categoricals.
- Engineer features that capture user behavior, product attributes, and context, drawing on industry practice for CTR prediction.
- Train and tune three ensemble models: Random Forest (bagging), Gradient Boosting, and XGBoost.
- Evaluate the models on a held-out test set and pick the best one.

Why this Approach

Ensemble tree models are the right choice for four reasons. First, the data is tabular with a mix of numeric, categorical, and time features. Tree models handle this mix natively, with no need for embedding layers. Second, CTR data is heavily imbalanced (around 94 percent non-clicks). Bagging and boosting both handle imbalance well with class weighting and sampling. Third, the dataset is small to medium (80,000 rows), which is the sweet spot for these models. Deep learning would need ten to a hundred times more data to win here. Fourth, the team needs interpretability. Tree models give clear feature importances and can be inspected with SHAP, which the product team can understand.

7. Solution Architecture

The solution will follow four clear steps, in order. Each step feeds into the next.

Step 1: Exploratory Data Analysis

- Join the five source tables on `user_id`, `product_id`, and `impression_id` to build the impression-level table.
- Check the shape of the combined data, column types, and missing values.
- Check the class balance: count of clicks vs non-clicks. Confirm the click rate is in the expected 5 to 7 percent range for personalized e-commerce recommendation slots.
- Study CTR by slot position, by page type, by category, by hour-of-day, by day-of-week, and by customer tier. These are the first-order drivers.
- Study CTR for cross-cuts: category by tier, slot position by device, page type by hour. These reveal the interactions a tree model will exploit.
- Look at price distributions, rating distributions, and the long-tail of brand and product IDs.

Step 2: Data Preprocessing

- Handle missing values: `age_band` missing for around 3 percent of users (impute with 'Unknown'), `device_at_impression` missing for around 1 percent of impressions (impute with the user's `primary_device`).
- Cap extreme outliers in price at the 99th percentile, since a handful of very expensive items distort feature distributions.
- Encode low-cardinality categoricals (`page_type`, `customer_tier`, `campaign_type`, `age_band`, `gender`, `city`, `state`, `device`, `category`, `ad_creative_version`) using one-hot encoding.
- For high-cardinality categoricals (brand has 80 levels, `product_id` has 1,200, `user_id` has 8,000), use target encoding (mean CTR for that level on the training set, with smoothing to avoid leakage on rare levels).
- Scaling is not needed for tree models, so skip it.
- Split the data into train and test using a time-based split (last 5 days held out as test). This mirrors how the model will actually be used.

Step 3: Feature Engineering

Raw columns alone do not capture the patterns that drive clicks. New features will be built from existing ones, following standard industry practice for CTR prediction:

- **user_ctr_x_category**: the user's past CTR for the product's category, captured from **user_history** and category affinity. Strong signal because shoppers click more on categories they engage with.
- **is_favourite_category**: 1 if the product category matches the user's favourite_category, else 0.
- **price_vs_aov_ratio**: product price (USD) divided by user's avg_order_value_usd. Captures whether the product is in the shopper's typical price range.
- **is_evening**: 1 if the impression hour is between 18 and 22 (6 PM to 10 PM), else 0. Evening shopping clicks more.
- **is_weekend**: 1 if day_of_week is Saturday or Sunday, else 0.
- **slot_position_squared**: captures the steep drop in CTR as slot position increases (positions 1 and 2 click much more than 7 and 8).
- **tenure_bucket**: short (less than 30 days), medium (30 to 365 days), long (more than 365 days). Captures customer maturity in three buckets that the team can reason about.
- **active_user_flag**: 1 if sessions_last_7d is greater than or equal to 5, else 0. Active users click more.
- **device_match**: 1 if device_at_impression equals primary_device, else 0. Captures whether the user is on their usual device.
- **discount_x_price**: an interaction term capturing that discounted items at certain price points click much better.

Step 4: Modelling with Ensemble Methods

Three models will be trained, tuned, and compared. All three are tree-based ensembles, chosen to cover the bagging family and the boosting family.

- **Random Forest (bagging)**: the bagging baseline. Robust, hard to overfit, easy to explain. Tune **n_estimators**, **max_depth**, and **min_samples_leaf** using 5-fold cross-validation on the training set. Use **class_weight='balanced'** to handle the imbalance.
- **Gradient Boosting (sklearn's GradientBoostingClassifier)**: the classical boosting model. Tune **n_estimators**, **learning_rate**, and **max_depth**. Compare against Random Forest to see whether the sequential boosting approach lifts performance.
- **XGBoost**: the modern boosting model and the industry standard for this kind of problem. Tune **n_estimators**, **learning_rate**, **max_depth**, **subsample**, **colsample_bytree**, and **scale_pos_weight** (to handle the class imbalance). Use early stopping on a validation set to pick the right number of trees.

For each model, generate feature importance and check it against business intuition. If a feature like 'user historical CTR' is not in the top five, something is wrong with the pipeline.

Step 5: Show How the Ranking Would Look for a User

Once the best model is picked, build a small demo that shows what the ranked list would look like for a single shopper. This makes the model concrete for the product team and is a useful sanity check before handover. The steps:

- Pick one user from the dataset (say a loyal customer with rich history).
- Fix a context: `page_type`, `slot_position`, `day_of_week`, `hour`, `device`.
- For every product in the catalog (1,200 products), build a feature row using that user, that product, and the fixed context.
- Score all 1,200 rows using the trained model.
- Sort by predicted click probability, descending. The top 8 products are what the model would put in the homepage carousel for this shopper in this context.
- Repeat the process for two or three different shoppers (one new, one regular, one loyal) and present the top-10 ranked lists side by side. The product team can eyeball whether the rankings look sensible.

Caution: this process shows how ranking would work, but it does not prove that the new ranking is actually better than today's static rules. The historical impression logs only tell us what happened when one product was shown in one slot, not what would have happened if a different product had been shown there. There is no way to validate the ranking offline. The only way to measure real improvement is to deploy the model and run an A/B test against the current rule-based system in production. That deployment and the A/B test are out of scope for this project and will be picked up by the engineering team as the next phase.

8. Risk, Mitigation, and Evaluation

Risks and Mitigations

Risk 1 (People): The product team does not engage with the model output. Mitigation: share feature importance and SHAP plots early. Walk the team through the model's predictions on a handful of real impressions before the handover.

Risk 2 (Process): The model performs well offline but the lift does not show up later in production. Mitigation: validate using a time-based split, not a random split, so the test set mimics how new data will arrive. Document the assumptions so the deployment team can test them in production.

Risk 3 (Platform / Data): Class imbalance distorts the model toward predicting 'no click'. Mitigation: use class weighting in Random Forest and Gradient Boosting, and `scale_pos_weight` in XGBoost. Evaluate using ROC-AUC, not accuracy, because accuracy is misleading on imbalanced data.

Risk 4 (Method): The model overfits to the small set of clicks. Mitigation: use cross-validation, regularisation (max_depth, min_samples_leaf, reg_alpha, reg_lambda), and a held-out time-based test set. Check that training and validation AUC are not far apart.

Evaluation Criteria

Because this is a binary classification problem with heavy imbalance, the primary evaluation metric is ROC-AUC on the held-out test set. ROC-AUC captures how well the model separates clickers from non-clickers across all probability thresholds, which is exactly the property we care about when the predictions will eventually be used to rank products. Accuracy is deliberately not used, because on data that is 94 percent non-clicks, a model that predicts 'no click' for everyone scores 94 percent accuracy and is useless.

Alongside ROC-AUC, two qualitative checks are part of the evaluation:

- Train vs test AUC gap: a large gap indicates overfitting. The gap should be small.
- Feature importance sanity check: the top features should match business intuition. User historical CTR, slot position, category match, and discount status should rank near the top. If they do not, the pipeline likely has a bug.

9. Expected Outcomes and Impact

Project Deliverables (end of this phase)

- A clean, joined impression-level training table built from the five source tables.
- An EDA report covering distributions, class balance, and the main first-order CTR drivers.
- A documented feature pipeline that takes raw data and produces the model-ready feature set.
- Three trained models (Random Forest, Gradient Boosting, XGBoost) with their ROC-AUC scores on the held-out test set, along with feature importance plots.
- A short handover note recommending which of the three models to take forward and why.

Downstream Outcomes (after deployment, out of scope here)

Once the model is deployed by engineering and tested in production through an A/B test, the business expects to see a measurable lift in CTR on personalized recommendation slots, a healthier mix of categories in each user's recommendations, and a foundation for more advanced work like conversion rate prediction and basket recommendations. None of this is measured in this phase. It is listed here so the team knows where the model is heading.

Why Now

Three things make this the right moment. First, competition from larger players (Amazon, Walmart, Target) is squeezing CTR on every channel, and the rule-based approach is no longer enough. Second, the company now has six months of clean impression logs, which is enough

data to train a useful first model. Third, rising customer acquisition costs (CAC across US digital channels has climbed every year since 2020) mean marketing spend is growing faster than revenue, and lifting on-site CTR is one of the few levers that improves revenue without spending more on ads.

Cost of Inaction

If the company keeps showing the same rule-based recommendations to everyone, three things will happen. CTR will keep drifting down as the rules become stale. Engaged shoppers will see less relevant products and slowly shift to platforms that personalize. Marketing spend per converted sale will keep rising, because acquisition is doing the work that personalization should be doing on-site. The cost is not dramatic in any single month, but it compounds quarter by quarter and the gap to better-personalized competitors widens.